ChatGPT’s Code:

```
#include <fstream>
#include <iostream>
#include <string>

class Sudoku {
private:
    int grid[9][9]{};
    int solutions = 0;

    bool canPlace(int row, int col, int value) const {
        for (int i = 0; i < 9; ++i) {
            if ((i != col && grid[row][i] == value) ||
                (i != row && grid[i][col] == value)) {
                return false;
            }
        }
        int startRow = (row / 3) * 3;
        int startCol = (col / 3) * 3;
        for (int r = startRow; r < startRow + 3; ++r) {
            for (int c = startCol; c < startCol + 3; ++c) {
                if ((r != row || c != col) && grid[r][c] == value) {
                    return false;
                }
            }
        }
        return true;
    }

    bool load(const std::string& filename) {
        std::ifstream file(filename);
        char cell;
        for (int row = 0; row < 9; ++row) {
            for (int col = 0; col < 9; ++col) {
                if (!(file >> cell) ||
                    (cell != '_' && (cell < '1' || cell > '9'))) {
                    return false;
                }
                grid[row][col] = (cell == '_') ? 0 : cell - '0';
```

```
            }
        }
        if (file >> cell) {
            return false;
        }
        return !file.bad();
    }

    void print() const {
        for (int row = 0; row < 9; ++row) {
            for (int col = 0; col < 9; ++col) {
                char cell = (grid[row][col] == 0)
                                ? '_' : static_cast<char>('0' +
grid[row][col]);
                std::cout << cell << (col == 8 ? '\n' : ' ');
            }
        }
        std::cout << '\n';
    }

    void solve(int position) {
        if (position == 81) {
            ++solutions;
            std::cout << "Solution " << solutions << ":\n";
            print();
            return;
        }
        int row = position / 9;
        int col = position % 9;
        if (grid[row][col] != 0) {
            solve(position + 1);
            return;
        }
        for (int value = 1; value <= 9; ++value) {
            if (canPlace(row, col, value)) {
                grid[row][col] = value;
                solve(position + 1);
                grid[row][col] = 0;
            }
        }
```

```
    }

public:

    bool run(const std::string& filename) {
        std::cout << "Puzzle file: " << filename << '\n';
        if (!load(filename)) {
            std::cout << "Error: File missing or invalid puzzle
format.\n\n";
            return false;
        }
        std::cout << "Original puzzle:\n";
        print();
        bool valid = true;
        for (int row = 0; row < 9; ++row) {
            for (int col = 0; col < 9; ++col) {
                if (grid[row][col] != 0 && !canPlace(row, col,
grid[row][col])) {
                    valid = false;
                }
            }
        }
        solutions = 0;
        if (valid) {
            solve(0);
        }
        if (solutions == 0) {
            std::cout << "No solution found\n\n";
        }
        return true;
    }
};

int main() {
    int status = 0;
    for (int number = 1; number <= 5; ++number) {
        Sudoku puzzle;
        if (!puzzle.run("puzzle" + std::to_string(number) + ".txt")) {
            status = 1;
        }
```

```
    }
    return status;
}
```

Gemini's Code:

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>

class SudokuCell {
private:
    int value;
    bool isFixed;

public:
    SudokuCell(int val = 0, fixed = false) : value(val), isFixed(fixed) {}

    int getValue() const { return value; }
    void setValue(int val) { value = val; }
    bool getIsFixed() const { return isFixed; }
    void setIsFixed(bool fixed) { isFixed = fixed; }
};

class SudokuBoard {
private:
    std::vector<std::vector<SudokuCell>> grid;
    std::string filename;

    bool isValid(int row, int col, int num) const {
        for (int i = 0; i < 9; ++i) {
            if (grid[row][i].getValue() == num) return false;
            if (grid[i][col].getValue() == num) return false;
        }
```

```
        int startRow = row - (row % 3);
        int startCol = col - (col % 3);
        for (int r = 0; r < 3; ++r) {
            for (int c = 0; c < 3; ++c) {
                if (grid[startRow + r][startCol + c].getValue() == num)
return false;
            }
        }
        return true;
    }

    bool solveRecursive(int row, int col, int& solutionCount) {
        if (row == 9) {
            solutionCount++;
            std::cout << "Solution " << solutionCount << " for " <<
filename << ":\n";
            printBoard();
            return false; // Return false to find all solutions, or true
if only one is needed. The instructions state: "if there is more than one
solution, your program should print them all out."
        }

        int nextRow = (col == 8) ? row + 1 : row;
        int nextCol = (col == 8) ? 0 : col + 1;

        if (grid[row][col].getValue() != 0) {
            return solveRecursive(nextRow, nextCol, solutionCount);
        }

        bool foundAny = false;
        for (int num = 1; num <= 9; ++num) {
            if (isValid(row, col, num)) {
                grid[row][col].setValue(num);
                if (solveRecursive(nextRow, nextCol, solutionCount)) {
                    foundAny = true;
                }
                grid[row][col].setValue(0);
            }
        }
        return foundAny;
```

```
    }

public:
    SudokuBoard() {
        grid.resize(9, std::vector<SudokuCell>(9));
    }

    bool loadFromFile(const std::string& fname) {
        filename = fname;
        std::ifstream infile(filename);
        if (!infile.is_open()) {
            return false;
        }

        for (int i = 0; i < 9; ++i) {
            std::string line;
            if (!(infile >> line)) return false;
            for (int j = 0; j < 9; ++j) {
                char ch = line[j];
                if (ch >= '1' && ch <= '9') {
                    grid[i][j] = SudokuCell(ch - '0', true);
                } else {
                    grid[i][j] = SudokuCell(0, false);
                }
            }
        }
        return true;
    }

    void printBoard() const {
        for (int i = 0; i < 9; ++i) {
            for (int j = 0; j < 9; ++j) {
                int val = grid[i][j].getValue();
                if (val == 0) std::cout << "_ ";
                else std::cout << val << " ";
            }
            std::cout << "\n";
        }
        std::cout << "\n";
    }
```

```
    void solve() {
        std::cout << "Puzzle File: " << filename << "\n";
        std::cout << "Original Puzzle:\n";
        printBoard();

        int solutionCount = 0;
        solveRecursive(0, 0, solutionCount);

        if (solutionCount == 0) {
            std::cout << "No solution found for " << filename << ".\n";
            std::cout << "----------------------------------------\n\n";
        } else {
            std::cout << "Total solutions found: " << solutionCount <<
"\n";
            std::cout << "----------------------------------------\n\n";
        }
    }
};

int main() {
    std::string puzzleFiles[5] = {"puzzle1.txt", "puzzle2.txt",
"puzzle3.txt", "puzzle4.txt", "puzzle5.txt"};

    for (int i = 0; i < 5; ++i) {
        SudokuBoard board;
        if (board.loadFromFile(puzzleFiles[i])) {
            board.solve();
        } else {
            std::cout << "Could not open file: " << puzzleFiles[i] <<
"\n\n";
        }
    }

    return 0;
}
```

GenAI Analysis:

For the GenAIs, I used ChatGPT and Gemini to write the code. I accessed both of them through their websites. For the prompt, I attached a pdf containing the following information, along with the words “create a program that follows these instructions”:

Assignment:

1. Use two GenAI’s to generate C++ code using objects, not functions, to implement the following requirements (Note: You may use either ChatGPT or CoPilot as one of your choices, but not both. They are essentially the same GenAI.): • Sudoku is a number puzzle in which a 9x9 grid is subdivided into 9 3x3 sub grids, or blocks.
   • A typical puzzle is shown here (image credit: Wikimedia Commons):

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

   - • Each of the cells in the puzzle must be filled in with a single digit 1-9, subject to the following constraints:
     - Each digit from 1 to 9 must appear exactly once in each row.
     - Each digit from 1 to 9 must appear exactly once in each column.
     - Each digit from 1 to 9 must appear exactly once in each block.
   - • Published puzzles are usually constructed to have a unique solution. This requires that at least 17 cells be filled in (and may require more; depending on the puzzle and which cells are empty, as many as 78 filled cells can be required to guarantee uniqueness). In general, the number of cells that are already filled in is not a particularly good indicator of how difficult a Sudoku puzzle is to solve.
   - • The Sudoku problem, then, is this: Given a partially completed Sudoku puzzle, find the (or at least a) solution; if no solution exists, prove that. • It is possible to solve a Sudoku puzzle entirely through logic. For example, in the above puzzle, consider the bottom right block, top right cell. Because of other numbers in the same row, column, or grid, that number cannot be 1, 2, 3, 5, 6, 7, 8, or 9, and so must be 4. By proceeding in this fashion, using these and more advanced deduction rules, it’s possible to gradually fill in the entire grid.
   - • That is not the approach we’re going to use. Instead, we’re going to implement a brute-force search, in which we exhaustively try possibilities until we find a solution. This will be slower to run but is easier to solve. (And by “slower to run,” we mean it'll finish in a few tenths of a second rather than a few milliseconds.)
   - • Here’s the idea: Suppose we have a Sudoku puzzle of which k of the 81 cells are unfilled. We identify an empty cell and deduce a list of candidates for what might go into that cell. If there are no such values (all digits are accounted for elsewhere in the same row, column, or grid), then we must have made a mistake in filling in the k cells and return a no-solution-found result. Otherwise, we try putting the first candidate value into the empty cell, and see if we can find a solution with the k+1 cells filled (i.e. a recursive call). If we get a no-solution-found to the first candidate, then we try the next, and so on, until we either find a solution (k == 81) or determine that no solution exists (all candidates report no solution, which again implies we took a wrong path somewhere earlier).
   - • This approach, by the way, is called depth-first search with backtracking. For each empty cell we make an attempt and see how far we can get with it. If we fail to find a solution, we back up until we find something else to try, and so on. If a solution exists, we are guaranteed to find it, because we exhaustively try all possibilities.
   - • You are given five files (puzzle1.txt through puzzle5.txt), each containing a Sudoku puzzle. Digits already filled in have been placed, and an underscore character (_) is used to indicate a blank square.
   - • Write a **recursive object-oriented** program to find the solution to the five puzzle files.
   - • The output for each solution, should include:
     - The puzzle file name (e.g., puzzle1.txt)
     - The puzzle from the file printed out.
     - The solution to the puzzle printed out or if no solution is found, “No solution found” printed out.
   - • The solution may not be unique; if there is more than one solution to the puzzle, your program should print them all out.
   - • To help you debug your program, solution1.txt is a solution to puzzle1.txt.

Reviewing both codes, the correctness of the programs differed heavily. Compiling ChatGPT’s code and using the test puzzles, it worked as intended and displayed the correct solution(s). However, Gemini’s code failed to compile in the first place because it did not establish the type of the variable “fixed.”

For time complexity, both of the programs use backtracking through depth first search as their implementation for the solution. Both of these would have O(9^N), scaling with the amount of empty cells, for the worst case. Regarding space complexity, ChatGPT used an integer array

to store the board. Gemini opted to use cell objects and nested vectors, which creates more space needed. The maintainability of ChatGPT's code is better, because the design of using one class is easier to understand, and Gemini failed to generate a functional program.

I decided to go with ChatGPT's code as the basis for the modification. I chose this option because it already has working code and a simpler design to read. Regarding code correctness, it remained the same as both the modified and original ChatGPT code output the same solutions. For execution time, ChatGPT suggested adding a while loop to skip over already filled cells. The previous program was causing unnecessary recursive calls which increased execution time by almost double of the amount for no reason. Skipping filled cells also reduces space as the less function calls free up more space on the stack. As for maintainability, I added comments throughout to explain what each code block/line is doing. ChatGPT also added SIZE and BLOCK_SIZE constants for better readability.